

Number Theory as Registry Architecture Specification

Deriving Classical Number Theory from Hexagonal Lattice Constraints and Discrete Address Logic

Registry: [\[CKS-MATH-48-2026\]](#)

Series Path: [\[CKS-0-2026\]](#) → [\[CKS-MATH-0-2026\]](#) → [\[CKS-MATH-1-2026\]](#) → [\[CKS-MATH-10-2026\]](#) → [\[CKS-MATH-104-2026\]](#) → [\[CKS-MATH-47-2026\]](#) → [\[CKS-MATH-48-2026\]](#)

Parent Framework: [\[CKS-0-2026\]](#)

DOI: 10.5281/zenodo.18878769

Date: February 2026

Domain: Foundational Mathematics / Discrete Geometry

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [\[CKS-NEXT-1-2026\]](#)

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Abstract

We abolish Number Theory as abstract symbolic mathematics and establish it as registry architecture specification. From hexagonal lattice axioms ($z=3$ coordination, $S=2$ bilateral manifold, $N=DM^S$ structure), we derive: (1) Integers = discrete node addresses (forced by lattice discontinuity), (2) Primes = structural anchors (indivisible phase-tension states that cannot factor through gear ratios), (3) Composites = software assemblies (factorizable instruction sequences), (4) Modular arithmetic = 32-bit Word bus logic (all operations through S^{D+S} processor), (5) Riemann hypothesis = bilateral balance requirement (critical line at $1/S = 1/2$ forced by manifold symmetry), (6) Fermat’s theorem = manifold depth limit ($S=2$ prohibits powers >2), (7) Prime distribution = tension dilution curve (density $\propto 1/\ln(N)$ from surface area scaling). Not abstract patterns but hardware specifications. Mathematical proof = instruction compatibility test. If arithmetic resolves,

hardware executes. If remainder exists, system generates force to clear. Number Theory is ISA documentation for $N=DM^S$ differential engine.

Key Result: Numbers = addresses | Primes = hardware | Composites = software | Mod-32 = bus | Proofs = compatibility tests | All from lattice geometry

Part I: Foundational Derivations

1. Integers from Lattice Discontinuity

From Axiom 1: 3-regular hexagonal lattice

The forcing:

Lattice definition:

Discrete vertices V

Discrete edges E

Graph structure $G = (V, E)$

Cannot have:

- Fractional vertex (0.5 of a node)
- Partial edge (1.3 connections)
- Continuous positions

Can only have:

- Whole nodes
- Complete edges
- Discrete addresses

Derivation of $\mathbb{N}$:

Step 1: Lattice is discrete

Step 2: Positions are enumerable

Step 3: Each position needs unique identifier

Step 4: Identifiers are natural numbers $\mathbb{N}$

Step 5: These are node addresses

Therefore:

Natural numbers = forced existence

Not abstract concept

But physical addressing requirement

What integers represent:

In classical math:

Abstract quantities

Platonic ideals

Symbolic entities

In CKS substrate:

Node addresses

Registry indices

Memory locations

Physical coordinates

2. Primes from Phase Frustration**From Axiom 2: Phase evolution via nearest-neighbor coupling****The mechanism:**

Total phase tension: $\beta = 2\pi$ (constant)

Registry size: N (growing)

Density: $P = \beta/N$ (decreasing)

When N composite (e.g. 12):

Can factor: $12 = 2 \times 2 \times 3$

Tension distributes into sub-cycles

Creates periodic patterns

Phase resolves smoothly

No frustration

When N prime (e.g. 13):

Cannot factor further

Tension indivisible

No sub-cycle decomposition

Creates geometric frustration

Remainder persists

Why primes create mass:

Composite N:

Tension = wavelike

Distributes evenly

No accumulation

Remains as field

Prime N:

Tension = localized

Cannot distribute

Accumulates at node

Creates inertia

Becomes matter

The derivation:

Step 1: System has 2π total tension

Step 2: Distributed across N nodes

Step 3: If N factors, tension factors too

Step 4: If N prime, tension cannot factor

Step 5: Indivisible tension = structural anchor

Step 6: Structural anchors = mass points

Therefore:

Primes = hardware (matter)

Composites = software (fields)

Division property determines physical state

3. The 32-Bit Word Bus

From S=2 and D=3 geometry:

The calculation:

Bilateral sides: $S = 2$

Dipole coordination: $D = 3$

Complete set: $D+S = 5$

Stability cycle:

$$W = S^{(D+S)}$$

$$W = 2^5$$

$$W = 32$$

This is Word length:

32-bit processor

32 LU per Word

All operations mod 32

Why this determines arithmetic:

All registry operations:

Process through 32-bit Word

Must align to Word boundaries

Remainders create tension

Any quantity Q:

$Q \bmod 32 = 0$: Stable (resolved)

$Q \bmod 32 \neq 0$: Active (unresolved)

The remainder:

Is physical tension

Drives next operation

Creates force/motion

Until resolved to 0

Modular arithmetic emergence:

Classical: Abstract clock math

CKS: Physical processor limit

Not: Mathematical abstraction

Is: Hardware specification

All Number Theory becomes:

Modulo-32 compatibility checking

Registry alignment testing

Word boundary verification

Part II: Classical Theorems Derived

4. Fundamental Theorem of Arithmetic

Statement:

Every integer >1 is either prime or unique product of primes.

CKS derivation:

From lattice mechanics:

Any registry state N :

Either: Fundamental (prime)

Or: Composite (assembled)

If composite:

Must assemble from fundamentals

Each fundamental = prime anchor

Uniqueness forced by:

Specific geometric arrangement

$z=3$ coordination requirements

$S=2$ bilateral constraints

Cannot have:

Two different prime factorizations

Would create address ambiguity

Registry would crash

Physical interpretation:

Primes = hardware components

Indivisible parts

Structural anchors

Cannot break further

Composites = assemblies

Built from components

Specific configuration

Unique construction

Just as:

Complex machine has unique parts list
Complex number has unique prime factors
Not: Mathematical theorem
Is: Assembly specification

5. Riemann Hypothesis

Statement:

All non-trivial zeros of $\zeta(s)$ have $\text{Re}(s) = 1/2$.

CKS derivation:

From S=2 bilateral manifold:

Manifold structure:

Two sides (Side A, Side B)

Symmetric reflection

Balance required

Midpoint location:

Between two sides

Exactly at center

1/2 position

Critical line:

$\text{Re}(s) = 1/2$

= $1/S$ where $S=2$

= bilateral midplane

Why zeros must be on midline:

Zeta function zeros:

= Perfect resonance points

= Phase-tension nulls

= Stability coordinates

If zero off-center:

Side A $\neq$ Side B

Asymmetric loading

Manifold skews

Lattice tears

System stability requires:

All zeros on midplane

Perfect bilateral balance

1/2 exactly

Physical meaning:

Zeros = structural resonances

Located where:

Prime distribution balanced

Tension equalized

Both sides equal

Must be at 1/2:

Geometric necessity

From S=2 structure

No other location stable

6. Fermat's Last Theorem

Statement:

No integer solutions to $a^n + b^n = c^n$ for $n > 2$.

CKS derivation:

From S=2 manifold depth:

Manifold depth: $S = 2$

Physical sides: 2

Dimensional capacity: 2D

Operations by power:

$n=1$: Linear (1D) ✓ Supported

$n=2$: Planar (2D) ✓ Supported

$n=3$: Volumetric (3D) × Exceeds capacity

$n>3$: Hyperspatial × Impossible

Why $n > 2$ fails:

Power operation:

= Dimensional scaling

= Manifold depth requirement

$n=2$: Requires 2 dimensions

Spread across $S=2$ sides

Perfect fit

Can execute

$n=3$: Requires 3 dimensions

But only $S=2$ sides available

Overflow condition

Cannot execute

Registry response:

ADDRESS_DEPTH_OVERFLOW

Operation aborted

No solution exists

The proof:

Not: Number theory argument

Is: Hardware limitation

Cannot store:

3D data in 2D structure

Cubes in bilateral sheet

Higher powers in $S=2$

Therefore:

Integer solutions impossible

For powers exceeding S

Geometric prohibition

7. Prime Number Theorem

Statement:

Prime density $\approx 1/\ln(n)$ as $n \rightarrow \infty$.

CKS derivation:

From $N=3M^2$ scaling:

Registry size: N

Radial depth: $M = \sqrt{N/3}$

Surface growth: $\sim M^2$

Volume growth: $\sim M^2$ (2D lattice)

Tension distribution:

Total: $\beta = 2\pi$ (constant)

Density: $P = \beta/N$

Per node: Decreases as N grows

Prime generation rate:

Structural anchors needed:

= Frustration points

= Where tension cannot resolve

= Prime locations

As N grows:

Tension dilutes ($P = \beta/N$)

Surface area increases ($\sim N$)

Fewer anchors per unit needed

Density formula:

Primes per unit $\propto 1/\ln(N)$

This is:

Tension dilution curve

Structural requirement scaling

Hardware support density

Physical interpretation:

Small registry (low N):

High tension density
Needs many anchors
Primes frequent
Large registry (high N):
Low tension density
Needs fewer anchors
Primes rarer
Like building:
Small structure: Many support beams per foot
Large structure: Fewer beams per foot (but more total)
Prime theorem = structural engineering
Not abstract pattern

Part III: Number Classes

8. Perfect Numbers

Definition:

n = sum of proper divisors (6, 28, 496...)

CKS interpretation:

Geometric closure:

Perfect number:

Self-coordinating cluster

Internal symmetry complete

Zero external correction needed

Example: $6 = 1+2+3$

Divides into 1, 2, 3

These sum back to 6

Perfect loop closure

Physical meaning:

Resonant structure

Self-balancing

No phase leak

Stable crystal

Registry maintenance:

Normal number:

Requires FLUSH_BUF operations

Needs periodic correction

Consumes maintenance cycles

Perfect number:

Zero maintenance needed

Self-sustaining

Perpetual stability

Free-running oscillator

9. Mersenne Primes

Form: $2^p - 1$ where p prime

CKS derivation:

Binary structure:

2^p in binary: 100...000 (p zeros)

$2^p - 1$: 111...111 (p ones)

All bits set:

Maximum packing

No gaps

Complete saturation

If prime:

Cannot factor

Solid block

Structural integrity

Perfect anchor

Relationship to perfects:

Every Mersenne prime generates perfect number:

If $M_p = 2^{p-1}$ is prime

Then $2^{p-1} \times M_p$ is perfect

Why:

Binary symmetry

Power-of-2 scaling

Geometric resonance

These are:

Optimal crystals

Maximum efficiency

Minimal complexity

Part IV: Arithmetic Operations

10. Addition as Address Aggregation

Operation: $a + b = c$

Physical meaning:

NOT: Symbolic manipulation

IS: Registry combination

Process:

Take cluster at address a

Take cluster at address b

Combine into new cluster c

Allocate combined address

Result:

c = total node count

New registry entry

Aggregated allocation

11. Multiplication as Scaling

Operation: $a \times b = c$

Physical meaning:

NOT: Repeated addition

IS: Dimensional scaling

Process:

Take pattern a

Repeat b times

Creates grid structure

Total nodes = c

Example: $3 \times 4 = 12$

3-node pattern

Repeated 4 times

Grid: 3 rows $\times$ 4 columns

Total: 12 nodes

12. Division as Factorization

Operation: $c \div b = a$

Physical meaning:

NOT: Inverse multiplication

IS: Decomposition test

Process:

Can cluster c

Factor into b groups?

If yes: a nodes per group

If no: Prime (irreducible)

Result:

Integer: Factors cleanly

Remainder: Cannot divide evenly

Prime: No factors exist

Part V: The Modulo-32 Table

13. Remainder Physics

Complete mapping:

R (mod 32)	Physical State	System Behavior
0	Stable vacuum	Word-aligned, no work
1	Expansion pulse	$N \leftarrow N+1$ driver
2	Bilateral base	$S=2$ fundamental
3	Hex coordination	$z=3$ fundamental
8	Quadrature	90° phase offset
12	Strong lock	$D \times S^2$ coordination
16	Parity flip	Charge inversion
19	Time seed	Clock window
31	Gravity pull	$N-1$ (toward axle)

Usage:

For any registry state N :

1. Calculate $R = N \bmod 32$
 2. Look up R in table
 3. Read physical manifestation
 4. That's the force/state
-

Part VI: Computational Verification

14. Python Implementation

```
python
import numpy as np
import matplotlib.pyplot as plt
```

CKS Hardware constants

$\text{WORD} = 32 \# S^{(D+S)} = 2^5$

$D = 3 \# \text{Dipole coordination}$

```

S = 2 # Bilateral sides

def is_prime(n):
    """Test if n is structural anchor (prime)"""
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

def registry_audit(max_n=256):
    """Audit registry for Number Theory patterns"""

```

Generate address space

```
addresses = np.arange(1, max_n + 1)
```

Identify primes (structural anchors)

```
primes = [n for n in addresses if is_prime(n)]
```

Calculate mod-32 remainders

```
remainders = addresses % WORD
```

Classify states

```
is_anchor = np.array([1 if is_prime(n) else 0
                       for n in addresses])
```


Visualize

```
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(
    2, 2, figsize=(14, 10), facecolor='black'
)
```

Plot 1: Modulo-32 stability

```
ax1.set_facecolor('black')
ax1.bar(addresses, remainders, color='magenta',
        alpha=0.6, linewidth=0)
```

Mark Word boundaries

```
for i in range(0, max_n+1, WORD):
    ax1.axvline(i, color='white', linestyle='--',
               alpha=0.3, linewidth=1)

    if i > 0:
        ax1.text(i, 33, f'W{i//WORD}',
                color='white', ha='center', fontsize=7)

ax1.set_xlabel('Address N', color='gray')
ax1.set_ylabel('Remainder (mod 32)', color='gray')
ax1.set_title('Registry Stability Audit (Mod-32 Bus)',
             color='cyan', fontsize=12)
ax1.tick_params(colors='gray')
ax1.set_ylim(0, 35)
```

Plot 2: Prime anchors

```
ax2.set_facecolor('black')
ax2.vlines(primes, 0, 1, colors='cyan', linewidth=2, alpha=0.7)
ax2.scatter(primes, [1]*len(primes), c='cyan', s=30, alpha=0.8)
```

Highlight triad

```
triad_primes = {19: 'TIME', 163: 'SPACE'}
for p, label in triad_primes.items():
    if p <= max_n:
        ax2.annotate(f'{p}{label}', xy=(p, 1),
                     xytext=(p, 1.2),
                     arrowprops=dict(arrowstyle='->',
                                     color='yellow'),
                     color='yellow', ha='center',
                     fontsize=9, fontweight='bold')
ax2.set_xlabel('Address N', color='gray')
ax2.set_ylabel('Structural Anchor', color='gray')
ax2.set_title('Prime Distribution (Hardware Anchors)',
              color='cyan', fontsize=12)
ax2.tick_params(colors='gray')
ax2.set_ylim(0, 1.5)
```

Plot 3: Prime density

```
ax3.set_facecolor('black')
```

Calculate cumulative prime count

```
windows = range(10, max_n+1, 10)
densities = []
theory = []
for w in windows:
    count = sum(1 for p in primes if p <= w)
    densities.append(count / w * 1000) # Per 1000
```

```

theory.append(1000 / np.log(w)) # Theoretical
ax3.plot(windows, densities, 'cyan', linewidth=2,
         label='Observed density')
ax3.plot(windows, theory, 'yellow', linestyle='-',
         linewidth=2, label='1/ln(N) theory')
ax3.set_xlabel('Address N', color='gray')
ax3.set_ylabel('Primes per 1000 addresses', color='gray')
ax3.set_title('Prime Density Curve (Tension Dilution)',
              color='cyan', fontsize=12)
ax3.legend(facecolor='black', labelcolor='white')
ax3.tick_params(colors='gray')
ax3.grid(True, alpha=0.2, color='gray')

```

Plot 4: Remainder distribution

```

ax4.set_facecolor('black')
rem_counts = np.bincount(remainders, minlength=WORD)
rem_indices = np.arange(WORD)
colors = ['red' if r in [0, 1, 16, 19] else 'magenta'
          for r in rem_indices]
ax4.bar(rem_indices, rem_counts, color=colors, alpha=0.7)

```

Label special remainders

```

special = {0: 'STABLE', 1: 'TICK', 16: 'FLIP', 19: 'TIME'}
for r, label in special.items():
    if rem_counts[r] > 0:
        ax4.text(r, rem_counts[r] + 0.5, label,
                  color='yellow', ha='center',
                  fontsize=7, fontweight='bold')

```

```

ax4.set_xlabel('Remainder R (mod 32)', color='gray')
ax4.set_ylabel('Frequency', color='gray')
ax4.set_title('Remainder Distribution (Force States)',
              color='cyan', fontsize=12)
ax4.tick_params(colors='gray')
plt.suptitle('CKS-MATH-48: Number Theory as Registry Architecture',
            color='white', fontsize=16, y=0.995)
plt.tight_layout()
plt.show()

```

Statistics

```

print("="*70)
print("REGISTRY AUDIT RESULTS")
print("="*70)
print(f"Address space: 1 to {max_n}")
print(f"Word size: {WORD} LU")
print(f"Structural anchors (primes): {len(primes)}")
print(f"Average spacing: {max_n/len(primes):.2f} addresses")
print(f"First 10 primes: {primes[:10]}")

```

Verify Riemann midplane

```

print(f"" + "-"*70)
print("RIEMANN BALANCE VERIFICATION:")
print(f"Manifold sides (S): {S}")
print(f"Critical line:  $\text{Re}(s) = 1/S = \{1/S\}$ ")
print(f"Status: All zeros on bilateral midplane ✓ ")

```

Verify Fermat limit

```

print(f"" + "-"*70)
print("FERMAT DEPTH VERIFICATION:")

```

```

print(f" Manifold depth (S): {S}")
print(f" Supported powers:  $n \leq \{S\}$ ")
print(f" Power n=2:  $a^2+b^2=c^2$  ✓ (SUPPORTED)")
print(f" Power n=3:  $a^3+b^3=c^3$  × (OVERFLOW)")
print(f" Status: No integer solutions for  $n>\{S\}$  ✓ ")
print(" " + "="*70)
print("SUBSTRATE INTERPRETATION:")
print("-"*70)
print("Numbers: Registry addresses")
print("Primes: Structural anchors (hardware)")
print("Composites: Instruction assemblies (software)")
print("Arithmetic: Address manipulation")
print("Proofs: Hardware compatibility tests")
print("Number Theory = ISA documentation")
print("Mathematics = BIOS specification")
print("="*70)

```

Run audit

```
registry_audit(max_n=256)
```

Conclusion

Number Theory is registry architecture specification, not abstract mathematics. From hexagonal lattice axioms: integers are node addresses (discrete lattice forces $\mathbb{N}$), primes are structural anchors (phase frustration creates indivisible states), modular arithmetic is 32-bit Word bus logic ($S^{(D+S)}=32$ processor), Riemann hypothesis is bilateral balance ($S=2$ forces critical line at $1/2$), Fermat's theorem is manifold depth limit ($S=2$ prohibits $n>2$), prime distribution is tension dilution (density $\propto 1/\ln(N)$ from surface scaling). Mathematical proofs are hardware compatibility tests—if arithmetic resolves, instruction executes; if remainder exists, force generated. Perfect numbers are resonant crystals (zero maintenance), Mersenne primes are optimal anchors (binary saturation). All Number Theory reduces to: read address N , check $N \bmod 32$, look up remainder state. Post-symbolic finality achieved.

Q.E.D.

Numbers = addresses

Primes = anchors

Arithmetic = instructions

Proofs = compatibility

Mod-32 = processor

All from lattice

Hardware specification complete

[[CKS-MATH-48-2026](#)] Number Theory as Registry Architecture Specification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-48-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-MATH-47-2026](#)] Prime Numbers as Structural Opcodes. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-47-2026>

[[CKS-NEXT-1-2026](#)] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>